

Use resources wisely. *Keep the answers useful.*

Understand the cost of context and computation,
then inspect what compression changes.

🕒 20-25 min core reading · 20-30 min practice · 12 short sections

Your movie assistant answers one person well. Now eight people ask at once, each with a long list of preferences and reviews. The same model can run out of memory or respond too slowly. Efficient LLM systems manage the cost of useful answers across weights, context, computation, and serving.

WHAT YOU WILL UNDERSTAND

The main question: which resource does an optimization save, and what must you check afterward?

The practical path: estimate weights and cache memory, inspect quantization error, and distinguish architectural changes from serving improvements.

Reading path: twelve short sections, three diagrams, two interactive experiments, and a Python workbook. Expand the optional notes for deeper mechanics.

1. Decide what efficient means

Efficiency needs a workload and a quality target. A movie assistant serving interactive chat has different priorities from a nightly job summarizing thousands of reviews. Measure useful completed work, not just the fastest isolated operation.

Measure	What it tells you	Movie-assistant example
Time to first token (TTFT)	Delay before the response starts	How long the user waits for the opening words
Time between output tokens	How quickly streaming continues	Whether the explanation arrives smoothly
End-to-end latency	Time until the answer finishes	Includes queueing, retrieval, and generation
Throughput	Work completed per unit time	Requests or output tokens served per second
Peak memory and cost	Resources needed at the chosen load	Whether concurrent requests fit economically

Report prompt/output lengths, concurrency, hardware, and model settings alongside results. A higher throughput can come with worse per-user latency. Check slower requests too: **p95 latency** is the value at or below which roughly 95% of measured request latencies fall.

▼ Go deeper: arithmetic is only part of the cost

An accelerator must move data as well as multiply numbers. A **compute-bound** workload is mainly limited by arithmetic capacity; a **memory-bandwidth-bound** workload is mainly limited by how fast data reaches that computation. Prefill has many known prompt positions to process together. Small-batch decoding often spends substantial time reading weights for relatively little work per step. These are tendencies, not rules for every model and device.

FLOPs count floating-point operations; **FLOP/s** is a rate. Fewer theoretical operations do not guarantee lower wall-clock latency if communication, conversion, or scheduling becomes the bottleneck. Benchmark the whole path with warmup, realistic lengths and load, and synchronized timing where the runtime executes device work asynchronously.

2. Count more than the model weights

The ideal weight payload is:

$$M_{weights} = P \frac{b_w}{8} \text{ bytes.}$$

P is parameter count and b_w is bits stored per parameter. Seven billion parameters at 16 bits require **14 billion bytes**, or **13.0385 GiB**. A GB is 10^9 bytes; a GiB is 2^{30} bytes. Use consistent units when comparing estimates with available memory.

Inference also needs cache state, activations, temporary workspaces, and runtime allocations. Quantized formats can add scales, zero points, padding, or higher-precision tensors. A weight-file size alone cannot establish whether the application fits.

➤ **Weights stay loaded. Sequence state grows.**

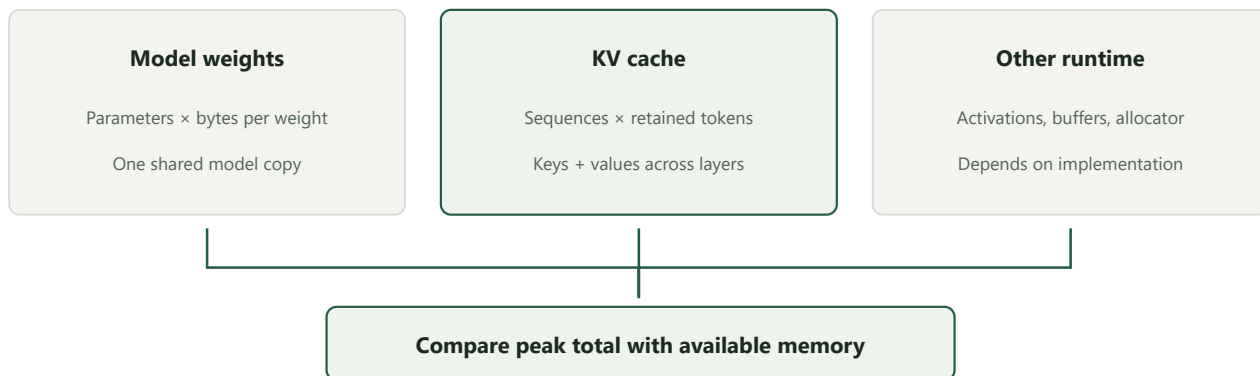


Figure 1. Inference memory has several parts. One model copy can serve multiple requests, but each distinct sequence needs cache state unless valid prefix sharing is used. Temporary buffers and runtime overhead also occupy memory; a weight-file size is not a complete capacity estimate.

For a conventional full-context transformer cache with equally long sequences:

$$M_{KV} = 2BTLH_{KV}d_h \frac{b_c}{8} \text{ bytes.}$$

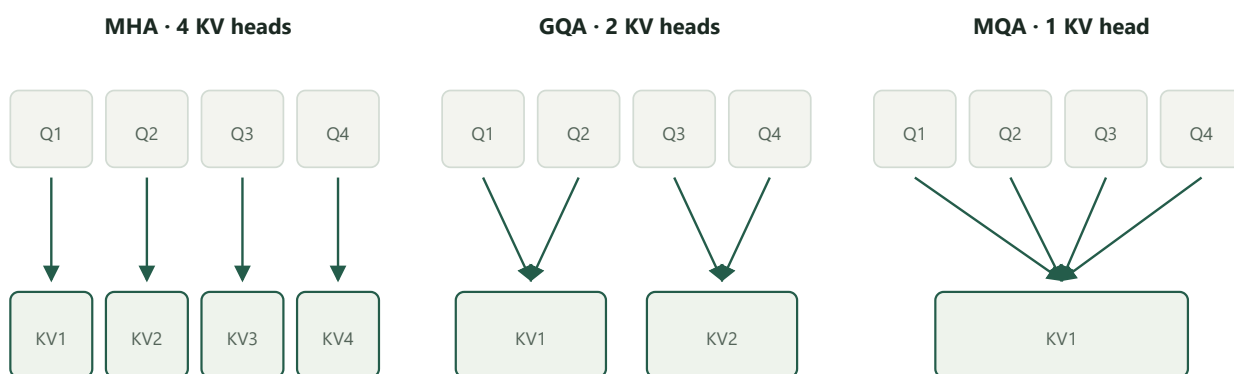
B is concurrent sequences, T is retained tokens per sequence, L is layers, H_{KV} is KV heads, d_h is head width, and b_c is cache bits per value. The factor 2 counts **both keys and values**. Distinct sequences share the model's weights, but ordinarily need their own cache entries.

With $B = 1$, $T = 4096$, $L = 32$, $H_{KV} = 8$, $d_h = 128$, and $b_c = 16$, the cache is **0.5 GiB**. Four such sequences need 2 GiB. Include room for generated tokens, not only the initial prompt. The formula assumes no prefix sharing, eviction, beam expansion, or cache compression beyond the specified precision.

3. Reduce cache cost with the right architecture

Chapter 24 introduced multi-head attention. In ordinary **multi-head attention (MHA)**, each query head has its own key and value head. **Grouped-query attention (GQA)** lets several query heads share a KV head. **Multi-query attention (MQA)** shares one KV head across all query heads. Queries still form their own attention scores and mixtures. The [GQA paper](#) explains this architectural tradeoff.

➤ **Several queries can share the same keys and values.**



Same number of query heads; fewer distinct cached KV heads.

The lab scales this idea to 32 query heads and compares 32, 8, or 1 KV heads.

Figure 2. A four-query illustration: MHA uses four KV heads, GQA shares two KV heads among query groups, and MQA shares one. Query heads still calculate their own scores and mixtures. These are architecture choices, not a guarantee that an existing checkpoint works after deleting KV heads.

In the previous memory example, switching the hypothetical KV-head count from 32 to 8 to 1 changes cache payload from **2 GiB** → **0.5 GiB** → **0.0625 GiB**. It does not establish equal answer quality or an identical runtime speedup. Deleting heads from a checkpoint is not a valid general conversion; use a compatible trained architecture or a tested adaptation method.

▼ **Go deeper: long context, sliding windows, and useful evidence**

With a full cache, retained length grows cache storage linearly. A new decoded query attends over that retained prefix. Dense prefill attention considers quadratically many position pairs, while the attention part of one new-token decode grows linearly with the retained length at fixed width.

Sliding-window attention limits the positions available to a layer and can bound that layer's retained cache. Some architectures mix local and global layers; their cache accounting must reflect both. Evicting tokens changes available context unless another mechanism preserves their information. Merely discarding old keys from a full-attention model is not an equivalent computation.

A supported long context window also does not prove reliable use of every detail. The [Lost in the Middle](#) study demonstrated sensitivity to where relevant information appears. Test your model with movie constraints placed throughout realistic inputs. Retrieval and concise context selection can still help when a larger window is available.

4. Store numbers with fewer bits

Quantization represents numerical values with a smaller set of values, reducing storage and sometimes computation or memory traffic. Weights, activations, and KV cache can use different precisions. A “4-bit model” does not automatically mean every operation and every tensor uses four bits. The [quantization concepts guide](#) describes these distinctions.

Our teaching quantizer uses symmetric integer codes. For b bits, set $Q = 2^{b-1} - 1$ and choose scale $s = \max |w| / Q$ for a group:

$$q = \text{clip}(\text{round}(w/s), -Q, Q), \quad \hat{w} = sq.$$

For 4 bits and maximum magnitude 1, $Q = 7$ and $s = 1/7$. A weight 0.43 becomes code 3 and reconstructs as **0.4286**. This scheme uses codes -7 through 7 , leaving -8 unused. The lost detail is rounding error; reconstruction does not recover the original value exactly.

One scale for many weights is cheap, but an outlier can make small values round poorly. Smaller groups add scales and can reduce error. Our six weights take an ideal 3 bytes of 4-bit codes; adding one 4-byte scale gives **7 bytes**, while three pair scales give **15 bytes**. Metadata matters, especially in tiny groups.

▼ Go deeper: calibration, PTQ, and quantization-aware training

Post-training quantization (PTQ) compresses an already trained model. Some methods use representative calibration examples to choose how to preserve important computations. [GPTQ](#) is an example of a more sophisticated weight-quantization method than independent rounding. Our workbook does not implement GPTQ.

Quantization-aware training (QAT) exposes training to simulated quantization effects so parameters can adapt. Integer quantization may use a scale and a zero point; low-bit floating-point formats use different representations. Storage precision, multiplication precision, and accumulation precision need not match.

Measure task quality and speed after conversion. Supporting a numeric format is not the same as having a fast kernel for your workload. Dequantization, unsupported operations, or transfer overhead can offset savings. Lower average weight error is useful diagnostic information, not a guarantee of a better recommendation.

5. Make attention move less data

Naive dense attention can materialize a large matrix of scores between query and key positions. **FlashAttention** tiles the computation and manages intermediate results so it avoids writing the full attention matrix to high-bandwidth memory. It calculates the same attention operation up to numerical differences, while reducing expensive memory traffic. See the [FlashAttention paper](#).

It does not remove the dense pairwise interactions: doubling sequence length still quadruples their count at fixed head count. **Sparse attention** instead limits which pairs interact and changes the visibility pattern. GQA reduces distinct KV heads; FlashAttention improves execution; sparsity changes connectivity. These solve different parts of the problem and can sometimes be combined.

Think of processing movie reviews: using a more efficient working layout differs from refusing to compare distant passages. Both may save resources, but only the second changes which passages can directly interact. Evaluate that change against the task.

6. Serve multiple requests without wasting capacity

Batching processes several requests together to reuse loaded weights and keep hardware busy. Waiting to assemble a larger batch can increase latency. With **continuous batching**, a server can admit new work as completed sequences leave, instead of waiting for an entire fixed batch to finish.

PagedAttention organizes cache memory in blocks, reducing waste from reserving large contiguous regions and supporting controlled sharing. It manages storage; it does not eliminate the keys and values a computation needs. The [PagedAttention paper](#) describes the approach.

Prefix caching reuses computed state for an identical compatible token prefix. Repeated movie-assistant instructions can benefit, but similar wording is not an exact cache match. Model state, tokenization, positions, and relevant execution settings must be compatible; a changed earlier prefix generally invalidates later cached state. Prefix reuse chiefly saves prompt processing, not the generation of a new answer. See [vLLM's prefix-caching documentation](#).

Compare these carefully: KV caching avoids recomputing earlier sequence state; prefix caching can reuse compatible state across requests; response caching reuses a finished answer. A response cache needs appropriate keys and freshness checks for changing movie availability.

7. Verify a cheap draft with the target model

Speculative decoding lets a cheaper draft mechanism propose several tokens, then asks the target model to verify them in parallel. Accepted tokens reduce the number of expensive sequential target passes. Poor acceptance or draft overhead can erase the benefit. The [speculative decoding paper](#) gives a distribution-preserving sampling algorithm.

For a greedy illustration, suppose the draft proposes "Moonlight Map is calm." The target agrees with "Moonlight Map" but next prefers "feels". Keep the matching prefix, append "feels", discard the remaining draft, and continue. Verification concerns token probabilities, not whether the recommendation is factually true.

➤ A draft proposes. The target model verifies.

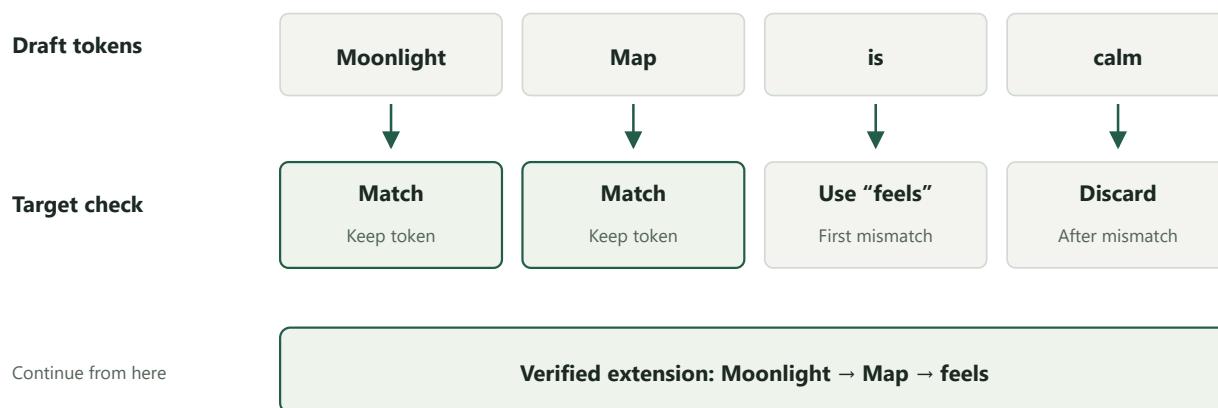


Figure 3. Illustrative greedy verification. The target agrees with the first two draft tokens, then prefers “feels” over “is”. Keep the matching prefix, use the target correction, and discard the remaining draft suffix. Distribution-preserving sampled speculation needs an acceptance and correction algorithm, not a simple text comparison.

▼ Go deeper: preserving the sampling distribution

For target distribution p and draft distribution q at a shared prefix, a draft token x is accepted with probability $\min(1, p(x)/q(x))$. On rejection, sample a correction from the normalized positive part of $p - q$ and discard the later draft suffix. If the entire draft is accepted, the algorithm can draw an additional target token.

This correction is essential: accepting only “plausible-looking” drafts does not preserve the target distribution. Exact distributional equivalence is not a promise of identical sampled text for the same seed, nor a promise of correct facts. The workbook does not implement speculative decoding.

8. Distinguish sparse computation from smaller models

A **mixture-of-experts (MoE)** layer contains several expert networks and a learned router that selects a subset for each token. Experts often replace feed-forward sublayers. They are numerical networks, not separate human specialists or guaranteed genre categories. Sparse activation reduces work per token relative to running every expert, while **total parameters** still include inactive experts. The [Switch Transformer paper](#) illustrates sparse expert routing.

If an illustrative layer has eight experts and activates two, that is one quarter of its expert paths per token, not automatically one quarter of the entire model’s cost. Shared layers, routing, uneven load, and communication remain. Inactive expert weights must still be stored or transferred when needed.

Distillation trains a student model to reproduce useful behavior from a teacher, for example from output distributions or generated demonstrations. The student can be smaller, but may lose capabilities. The [original distillation paper](#) explains learning from teacher predictions. **Pruning** removes selected weights or structures; speed improves only when the runtime can exploit the resulting structure or sparsity.

These differ from spending more computation at inference time. Generating longer deliberations, testing candidate answers, or sampling several alternatives can improve some tasks while increasing latency and cost. More computation alone does not establish correctness.

9. Match the system to its hardware and workload

Strategy	What is distributed or moved	Main tradeoff
Replicas	Separate model copies handle separate requests	More aggregate capacity, but duplicate weights
Tensor parallelism	Parts of a layer across devices	Frequent communication between devices
Pipeline parallelism	Different layers or stages on different devices	Scheduling gaps and stage balance
Expert parallelism	MoE experts across devices	Token routing and load balance
CPU/disk offloading	Some state moved out of accelerator memory	Extra transfers can slow execution

Multiple devices do not automatically divide every memory term evenly or halve latency. For example, [tensor-parallel inference](#) needs communication while combining partial layer results. The interconnect can become a bottleneck.

For the movie assistant, first identify the problem. Long review bundles can inflate prefill and cache costs; many simultaneous users can strain serving capacity; an unnecessarily long answer can waste decode time. Choose a change, measure against the same requests, and rerun recommendation-quality checks. A deployment decision needs both resource evidence and useful answers.

▼ Go deeper: inference memory is not a training budget

Training also stores gradients, optimizer state, and activations needed for backward computation. **Activation checkpointing** saves memory by recomputing selected activations during backpropagation. It trades extra computation for less stored state; it is different from saving a model checkpoint to disk.

Sharding training state can distribute parameters, gradients, or optimizer buffers. Parameter-efficient fine-tuning changes which parameters are trained, but does not necessarily remove the need to run the base model. Do not reuse this chapter's inference calculator as a training capacity estimate.

For a serving comparison, record model and tokenizer versions, precision, context lengths, concurrency, cache reuse, software, and hardware. Measure peak memory, TTFT, completion latency, throughput, and task results under the intended workload. A single average from a short prompt is insufficient evidence for a busy service.

10. Explore memory and rounding error

Start with the default 16 GiB budget. Increase KV heads to 32, then return to 8 and raise concurrency. Reduce weight precision while leaving cache precision unchanged. Notice which terms move. In the second view, inspect the weight 0.43, then try the outlier with one scale and with pair scales.

COUNT. COMPRESS. COMPARE.

Calculated estimates · no GPU run

Smaller numbers. Different tradeoffs.

Estimate memory for a hypothetical movie assistant, then inspect quantization on six hand-set weights. These are separate teaching calculations, not a hardware benchmark.

Default memory estimate: 7B parameters, 16-bit weights, 32 layers, 8 KV heads of width 128, 16-bit cache, one 4096-token sequence, and a 16 GiB budget.

Weights	KV cache	Reserve	Total
13.0385 GiB	0.5000 GiB	2.0000 GiB	15.5385 GiB

Headroom = **0.4615 GiB**. With 32 KV heads, cache becomes 2 GiB and total becomes 17.0385 GiB. Weight-only 4-bit payload reduces default total to 5.7596 GiB. The reserve is illustrative; these figures do not certify that a real model fits.

Default quantizer: 4-bit symmetric codes, one scale = 1/7. Fixed features = [0.2, 0.6, 0.1, 0.3, 0.8, 1].

Original	Code	Restored	Absolute error
-1.0000	-7	-1.0000	0.0000
-0.4600	-3	-0.4286	0.0314
-0.1000	-1	-0.1429	0.0429
0.0800	1	0.1429	0.0629
0.4300	3	0.4286	0.0014
1.0000	7	1.0000	0.0000

Mean weight error = **0.0231**. Matching score changes from **0.8820 to 0.9143**. Packed-layout estimate = 3 payload bytes + 4 scale bytes = 7 bytes, versus 24 bytes at float32. None of these quantities is task accuracy or measured speed.

The memory planner holds total parameter count fixed to isolate cache effects. Real architecture changes may change that count. Its fixed 2 GiB reserve is illustrative, and payload counts exclude quantization metadata. “Within estimate” is not a device-fit guarantee. The six-weight quantizer is independent of the hypothetical 7B model; no LLM is loaded, trained, or benchmarked.

The four-file workbook uses Python 3.10+ and the standard library, with no network calls. It contains `data.json`, `efficiency.py`, `test_efficiency.py`, and `README.md`.

```
py efficiency.py
py efficiency.py --weight-bits 4 --batch 4 --tokens 8192
py efficiency.py --kv-heads 32 --budget 16
py efficiency.py --preset outlier --quant-bits 4 --grouping pairs
py -m unittest -v test_efficiency.py
```

Use `python` or `python3` if needed. Each calculation prints a JSON report for both experiments; invalid CLI choices exit with code 2. Reports retain full precision while screen values are rounded.

Exercise 1: estimate cache growth

The default cache uses 0.5 GiB for one sequence of 4096 tokens. What happens with four sequences of 8192 tokens? Do the weights also grow eightfold?

▼ Reveal the reasoning

Cache grows by $4 \times 2 = 8$, to 4 GiB, under the full-cache assumptions. One shared weight copy stays at 13.0385 GiB. Adding the illustrative 2 GiB reserve gives 19.0385 GiB, above the 16 GiB budget.

Exercise 2: include the quantization metadata

Six weights use four-bit codes. What is the ideal storage with one four-byte scale? With a scale per pair? Why is either number different from the size of the JSON report?

▼ Reveal the reasoning

Codes need $6 \times 4 / 8 = 3$ bytes. One scale gives $3 + 4 = 7$ bytes; three scales give $3 + 12 = 15$ bytes. Those estimates assume packed codes and omit alignment and headers. JSON stores textual numbers and field names; it is a readable report, not that binary representation.

Exercise 3: choose the right quality check

For the outlier example, one scale gives mean weight error 0.1912 and score 6.0000. Pair scales reduce mean error to 0.0774 but give score 6.2443. The original score was 5.8820. Which score is closer, and what should you conclude?

▼ Reveal the reasoning

The one-scale score is closer: its absolute error is 0.1180, versus approximately 0.3623 for pair scales. Different signed errors can cancel or reinforce after multiplication by input features. Lower mean weight error does not guarantee a better output. These invented scores are not recommendation accuracy; evaluate a compressed assistant on the real task.

11. Check your understanding

01 A model's weight payload fits in GPU memory. What can you conclude?

- A. Only that the weights fit; cache and runtime allocations still need room.
- B. Every supported context length and batch size will fit.
- C. The model must generate tokens faster than a smaller model.

Answer: A. Inference also needs KV state, activations, temporary buffers, and runtime allocations. Real peak memory and latency require measurement under the intended workload.

02 With all other settings fixed, what happens when retained tokens per sequence double?

- A. Weights and KV cache both double.
- B. The full-context KV payload doubles; weight payload stays fixed.
- C. Only the number of model parameters changes.

Answer: B. The cache formula is linear in retained length under this chapter's full-cache assumptions. Weights are shared across positions; sliding-window or compressed-cache designs need their own accounting.

03 What does FlashAttention primarily improve for dense attention?

- A. It removes every interaction between distant positions.
- B. It replaces the transformer with a recurrent model.
- C. It reduces memory traffic using a different implementation of the attention computation.

Answer: C. Tiling avoids materializing the full score matrix in high-bandwidth memory. Dense attention still has quadratic pairwise work in sequence length, up to implementation and numerical details.

04 A mixture-of-experts model activates two experts per token. Which statement is correct?

- A. Only two experts' weights exist in the entire model.
- B. Active computation can be sparse while total stored parameters remain much larger.
- C. Each expert is guaranteed to represent a human-readable topic.

Answer: B. A learned router selects experts for a token. Inactive experts still belong to the model, and storing or moving them costs resources. Expert roles are learned, not guaranteed topic labels.

05 Pairwise quantization reduces average weight error but increases one movie score's error. Is that possible?

- A. No; smaller weight error always means a more accurate output.
- B. No; quantization changes storage only, never values.
- C. Yes; feature weighting and error cancellation affect outputs, so evaluate the task.

Answer: C. The output depends on the signed errors weighted by its input features. Mean absolute weight error loses that information. A real recommendation system needs task-level checks after compression.

12. Choose how to adapt the assistant

You can now distinguish changes to **numeric precision, cached state, attention execution, serving schedules, architecture, and hardware placement**. Each targets a different cost. Keep the measurement separate from the estimate, and the resource saving separate from the quality check.

Chapter 27: Choosing between prompting, RAG, tools, and fine-tuning starts the next part of the handbook. It will help you choose how to change an assistant's behavior or supply knowledge before investing in a more complicated system.

▼ Quick-reference glossary

Term	Meaning
TTFT / throughput / p95	Delay to first token / work per time / 95th-percentile latency
GB / GiB	10^9 bytes / 2^{30} bytes
KV cache / GQA / MQA	Stored attention state / grouped KV sharing / one shared KV head
Quantization / scale	Reduced numerical representation / conversion between codes and values
PTQ / QAT	Quantize after training / train with quantization effects
FlashAttention / sparse attention	Reduce attention memory traffic / restrict interacting positions
Continuous batching / PagedAttention	Update active serving batches / manage cache in blocks
Prefix cache / response cache	Reuse compatible prefix state / reuse a completed answer
Speculative decoding	Propose cheap draft tokens and verify with the target model
MoE / distillation / pruning	Route among experts / train from a teacher / remove selected structure
Tensor / pipeline / expert parallelism	Split layers' calculations / split stages / split experts
Offloading / activation checkpointing	Move state out of device memory / recompute activations in training

Sources and further reading

Primary references are linked beside the concepts: GQA, FlashAttention, PagedAttention, speculative decoding, Switch Transformers, distillation, GPTQ, Lost in the Middle, and the Hugging Face and vLLM documentation. Movie examples, diagrams, capacity assumptions, and numerical exercises are original teaching material. No performance benchmark is claimed for the workbook.